

helixqa_integration_sec06

6. Phase 5: Catalogizer Example Integration

This chapter applies the HelixQA integration methodology to the Catalogizer multi-platform media management system — a production-scale target with five client applications, 39 Git submodules, and an OCU (OpenClaw Ultimate) autonomous QA pipeline. The analysis proceeds in four stages: current-state assessment, submodule bump and synchronization, new test bank creation, and client-application validation. Every command, file path, and commit hash is extracted from the live repository state as of 2026-04-30.

6.1 Current State Assessment

6.1.1 HelixQA at 35deb43 — Two Phases Behind Upstream

Catalogizer pins HelixQA at commit 35deb43 (Phase 27.7, “visionnav Provider interface + NopProvider”), landed via parent commit e1307bd with the message `chore(submodules): bump HelixQA to 35deb43 (Phase 27.7 visionnav Provider...) 1`. Upstream main has advanced to 0bca023 (Phase 29, “§12.6 Memory-Budget Ceiling”), a two-phase delta ^[2]. Phase 29 introduces a 60 % RAM cap (`HOST_SAFETY_MAX_MEM_PCT=60`) enforced through `bounded_run()` scope wrapping, three new pre-build covenant gates (`CM-COVENANT-CONSTITUTION-126`, `CM-MEM-COVENANT-PROPAGATION`, `CM-MEMBUDGET-METATEST`), and propagation of the §12.6 clause to 39 governance files across all submodules ^[2]. Catalogizer therefore lacks both the runtime safety protections and the governance synchronization Phase 29 mandates. The triggering incident was three consecutive session-loss SIGKILLs during an unbounded `m -j 5` AOSP build on a 62 GB host ^[2]; without the `bounded_run()` wrapper, a parallel Full-QA Master Cycle exposing 60+ banks to the same failure mode.

`DocProcessor` (5f1e58a), `LLMOrchestrator` (1b95823), and `LLMProvider` (0720b6e) are at their latest upstream commits ^[3]^[4]^[5]. `VisionEngine` is unpinned — no commit hash is recorded in `.gitmodules`, and the latest upstream is 77765aef ^[6]. The bump strategy therefore focuses on HelixQA while treating the remaining HelixDevelopment submodules as a lighter synchronization pass.

6.1.2 Existing QA Artifacts: 60+ Banks, 5 Session Archives, 206 PASS / 1 SKIP

Catalogizer maintains QA archives under `docs/reports/qa-sessions/` containing five dated sessions: 2026-04-18-T2158 (Full-QA Master Cycle baseline), 2026-04-21-T (KeyPress Android-9 fallback), 2026-04-21-T-v2 (Run5 ticket-persistence triage), 2026-04-22-T16-28 (auto-committed session), and 2026-04-22-T17-16/helixqa (Android TV challenge validation closure) ¹. Each session holds a `FINAL-REPORT.md`, `logs/`, `challenges/` JSON results, `helixqa/` bank and autonomous results, `videos/` MP4 recordings, `screenshots/` pre/post per action, and `tickets/` markdown tickets with evidence ¹.

The most recent audit baseline at commit `f9c7781` reports 0 FAIL / 206 PASS / 1 SKIP [1](#). The single SKIP is a constitutionally-mandated exclusion: `full-qa-android` and `full-qa-androidtv` banks are SKIPPED in autonomous mode because the only available ADB devices are ATMOSphere instances, permanently excluded per `.devignore` under Constitution Article VII §7.1 [1](#). The 206 PASS count covers non-Android banks (API, web, desktop, cross-platform) and internal HelixQA unit tests. This baseline is the regression checkpoint against which any bump must be compared.

6.1.3 Integration Gaps: Android Blocked, Installer-Wizard and API Client Uncovered, CUDA Sidecar Not Deployed

Three critical deficiencies persist. First, Android and Android TV autonomous QA is a **fatal blocker** — the `full-qa-android` and `full-qa-androidtv` banks exist but cannot execute because no non-ATMOSphere Android device is connected via `adb connect` [1](#). Second, the `installer-wizard` (Tauri-based SMB configuration wizard) and `catalogizer-api-client` (TypeScript library distributed as `@catalogizer/api-client`) have **no identified HelixQA bank coverage**; they rely on manual testing and internal unit-test suites (`installer-wizard` claims 93 % coverage with 30/30 tests passing) [1](#). Third, `OCU-CUDA-Sidecar/` exists in the repository but the sidecar container is **not deployed** on `thinker.local`, leaving NVENC encoding and CUDA-accelerated vision inference stubbed rather than exercised [1](#).

Table 1 consolidates these findings.

Table 1. Catalogizer HelixQA Integration — Current State Assessment

Item	Current Status	Gap	Target State
HelixQA submodule	Pinned at <code>35deb43</code> (Phase 27.7)	Missing Phase 28–29 (memory budget, covenant propagation, 3 new pre-build gates)	Pinned at <code>0bca023</code> (Phase 29) with all gates green
DocProcessor submodule	<code>5f1e58a</code> (latest)	None	Maintain at tip
LLMOrchestrator submodule	<code>1b95823</code> (latest)	None	Maintain at tip
LLMProvider submodule	<code>0720b6e</code> (latest)	None	Maintain at tip
VisionEngine submodule	Unpinned (no commit)	No version lock; latest is <code>77765aef</code>	Pin to <code>77765aef</code> or latest verified tip
Existing bank suite	60+ banks, 206 PASS / 1 SKIP	Cannot assess Phase 29 behavior until bump completes	206 PASS / 0 SKIP or documented SKIP rationale
Android autonomous QA	<code>full-qa-android</code> bank exists but SKIPPED	<code>.devignore</code> excludes	Connect non-ATMOSphere phone; add to <code>.devconnect</code>

Item	Current Status	Gap	Target State
		ATMOSphere ADB devices	
Android TV autonomous QA	full-qa- androidtv bank exists but SKIPPED	Same ADB device exclusion as phone	Connect non- ATMOSphere Android TV; add to .devconnect
Installer-wizard coverage	30/30 unit tests passing, 93 % claimed	No HelixQA bank; no E2E coverage	catalogizer- desktop-e2e bank covering wizard launch and SMB discovery
API client coverage	Vitest + Jest dual config	No HelixQA bank; no contract tests against real HTTP	catalogizer-api- contract bank with real endpoint calls
CUDA sidecar deployment	OCU-CUDA- Sidecar/ exists in tree	Not deployed on thinker.local	Docker image built and running; NVENC path exercised

The table reveals that the Catalogizer–HelixQA integration is structurally mature — the framework is wired, orchestration scripts exist, and the baseline is stable — but operationally incomplete in four dimensions: version currency, device availability, client coverage, and sidecar deployment. The bump to Phase 29 is the prerequisite for all subsequent work because the new memory-budget protections affect how the Full-QA Master Cycle schedules parallel bank execution.

6.2 Bump and Synchronization Steps

The bump procedure assumes Go 1.24, GNU Make 4.3+, and Docker Engine 25.x or Podman 4.x, matching the build environment in `catalog-api/ARCHITECTURE.md` and the `containers/` submodule documentation [1](#).

6.2.1 Step 1: Update HelixQA to Latest Upstream

From the Catalogizer repository root, execute a single command to fetch the upstream HelixQA main branch and advance the submodule pointer.

```
cd /path/to/Catalogizer
git submodule update --remote HelixQA
```

The `--remote` flag fetches `git@github.com:HelixDevelopment/HelixQA.git` and checks out the tip commit, `0bca023` at the time of writing ^[2]. After execution, `git -C HelixQA log --oneline -1` must emit `0bca0234bab37b5ebdb28e22fd04350b51847883`. The parent working tree now shows HelixQA as modified. Do not commit yet; verification must complete first.

6.2.2 Step 2: Verify HelixQA Builds and Tests

Enter the submodule and run its build and test targets.

```
cd HelixQA
make build
make test
```

Expected output for `make build`: all `cmd/` binaries compile without `go vet` or `gofmt` errors. Expected output for `make test`: 49 anti-bluff-compliant tests PASS across `pkg/audio` (17 tests), `cmd/qa-audio-probe` (7 tests), and `pkg/visionnav` (25 tests) ^[2]. The Phase 29 pre-build gates — `CM-COVENANT-CONSTITUTION-126`, `CM-MEM-COVENANT-PROPAGATION`, `CM-MEMBUDGET-METATEST` — must report green. The `CM-MEMBUDGET-METATEST` gate runs `scripts/testing/test_memory_budget_covenant.sh` and validates 15 invariants, including `HOST_SAFETY_MAX_MEM_PCT default = 60` and `build.sh` wraps `m-j` in `bounded_run` scope ^[2].

6.2.3 Step 3: Synchronize Remaining HelixDevelopment Submodules

`DocProcessor`, `LLMOrchestrator`, and `LLMProvider` are already at their latest commits. `VisionEngine` is unpinned and must be locked to a known-good commit.

```
git -C DocProcessor log --oneline -1      # expect 5f1e58a
git -C LLMOrchestrator log --oneline -1   # expect 1b95823
git -C LLMProvider log --oneline -1       # expect 0720b6e
git -C VisionEngine log --oneline -1      # may be empty or
detached
```

If `VisionEngine` reports no commit, initialize and pin it:

```
git submodule update --init VisionEngine
cd VisionEngine
git checkout 77765aef6d103a3d2b3358df8007205f5b35143a
cd ..
git add VisionEngine
```

`VisionEngine` is a computer-vision/OCR engine consumed by `HelixQA` for autonomous QA screenshot analysis, not a runtime dependency of the media server, but pinning ensures reproducibility of the QA toolchain.

6.2.4 Step 4: Run the Catalogizer Full-QA Master Cycle

With `HelixQA` at Phase 29, trigger the complete QA campaign through the top-level `Makefile` entry point, which delegates to `scripts/helixqa-orchestrator.sh` ¹.

```
cd /path/to/Catalogizer
make qa-all
```

The `qa-all` target executes the 10-phase Full-QA Master Plan mandated by `docs/plans/2026-04-18-full-qa-cycle-master-plan.md` under Constitution Article VII (§7.1–§7.11) ¹: (1) governance and session directory creation, (2) clean rebuild of all apps and services, (3) unit and integration tests for every submodule, (4) challenges bank run, (5) `HelixQA` bank tests for non-Android platforms, (6) `HelixQA` autonomous QA for non-Android platforms, (7) video and screenshot post-session review, (8) fix loop with root-cause analysis and regression tests, (9) version bump and release artefact generation, and (10) final session report assembly ¹.

Non-Android banks (full-qa-api, full-qa-web, full-qa-desktop, full-qa-cross-platform) execute against the running stack brought up by docker-compose.qa.yml and docker-compose.qa-robot.yml [1](#).

6.2.5 Step 5: Verify No Regressions Against the 206 PASS / 1 SKIP Baseline

The final step is strict regression comparison against the f9c7781 baseline (0 FAIL / 206 PASS / 1 SKIP) [1](#). Acceptable outcomes: zero new FAILs (any new failure indicates a Phase 28–29 breaking change requiring triage before commit), zero unexpected SKIPS (the single Android SKIP is constitutionally mandated and expected), and PASS count ≥ 206 (new banks may increase the count, but it must never decrease).

Table 2 enumerates the bump steps with exact commands, expected outputs, and verification criteria.

Table 2. HelixQA Bump Procedure — Commands, Expected Output, and Verification

Step	Command	Expected Output	Verification
1. Fetch latest HelixQA	<code>git submodule update --remote HelixQA</code>	Submodule advances to 0bca023; git status shows HelixQA modified	<code>git -C HelixQA log --oneline -1</code> returns 0bca023
2. Build HelixQA	<code>cd HelixQA && make build</code>	All cmd/ binaries compile; go vet clean; gofmt -l empty	Exit code 0; no vet/format errors
3. Test HelixQA	<code>cd HelixQA && make test</code>	49/49 PASS across pkg/audio, cmd/qa-audio-probe, pkg/visionnav; 3 pre-build gates green	<code>echo \$?</code> returns 0; gate logs show PASS
4. Sync VisionEngine	<code>git submodule update --init VisionEngine && cd VisionEngine && git checkout 77765aef</code>	VisionEngine pinned to 77765aef; others unchanged	<code>git -C VisionEngine log --oneline -1</code> returns 77765aef
5. Run Full-QA Master Cycle	<code>cd .. && make qa-all</code>	10-phase plan executes; non-Android banks run against Docker QA stack	Session directory created under docs/reports/qa-sessions/ with timestamp
6. Regression comparison	<code>cat docs/reports/qa-sessions/*/FINAL-REPORT.md</code>	PASS ≥ 206 ; FAIL = 0; SKIP ≤ 1 (Android only)	Manual review against f9c7781 baseline; operator signs off
7. Commit bump	<code>git add HelixQA VisionEngine && git commit -m "chore(submodules): bump HelixQA to</code>	Commit hash generated; tree shows updated submodule pointers	<code>git log --oneline -1</code> contains 0bca023 and Phase 29

Step	Command	Expected Output	Verification
	0bca023 (Phase 29) + pin VisionEngine"		

The procedure is sequential because each step gates the next. Running `make qa-all` before `make test` in HelixQA risks propagating a broken upstream commit into the baseline. The regression comparison in Step 6 is not fully automated — it requires operator judgment because the cycle generates video recordings, screenshots, and ticket artefacts that a numeric diff cannot assess.

6.3 New Test Bank Creation

The existing 60+ HelixQA banks cover generic Catalogizer surfaces (API, web, desktop, Android, Android TV) and OCU program phases (P0–P7) [1](#). What they do not cover are the two client applications lacking any HelixQA bank — `installer-wizard` and `catalogizer-api-client` — and the translation-specific workflows (subtitle management, Cyrillic/Unicode handling, multilingual language settings). This section defines five new banks and provides two complete YAML implementations.

6.3.1 Web Functional Bank: `banks/catalogizer-web-functional.yaml`

This bank covers the Collection-Manager-React web application’s primary user journeys: login, media browsing, search, favorites management, and settings modification. The web client is built with Vite, Tailwind CSS, and React 18; it connects to the Go backend via `Catalogizer-API-Client-TS` and receives real-time updates over `WebSocket-Client-TS` [1](#). The bank asserts both HTTP REST correctness and WebSocket state propagation.

```
# SPDX-FileCopyrightText: 2026 Milos Vasic
# SPDX-License-Identifier: Apache-2.0

version: "1.0"
name: "Catalogizer Web Functional Test Bank"
description: "End-to-end functional tests for the Collection-
             Manager-React web client"
metadata:
  author: "vasic-digital"
  app: "Catalogizer"
  version: "2.3.0"
  client: "catalog-web"
  build_tool: "vite"

test_cases:
  - id: WEB-001
    name: "User login with valid credentials"
    category: functional
    priority: critical
    platforms: [web]
```

- steps:
- name: "Navigate to login page"
action: "navigate /login"
expected: "Login form rendered with email and password fields"
 - name: "Enter credentials"
action: "input [data-testid=login-email] user@example.com; input [data-testid=login-password] secret"
expected: "Both fields populated"
 - name: "Submit form"
action: "click [data-testid=login-submit]"
expected: "200 OK from POST /api/v1/auth/login; JWT stored in localStorage"
 - name: "Verify redirect"
action: "waitForNavigation"
expected: "URL is /dashboard; auth context populated"
- tags: [auth, login, web]
estimated_duration: "15s"
expected_result: "Authenticated session established; WebSocket connection opened"
- id: WEB-002
name: "Media browsing with pagination"
category: functional
priority: critical
platforms: [web]
steps:
- name: "Load media grid"
action: "navigate /media"
expected: "GET /api/v1/media?page=1 returns 200 with items array"
 - name: "Scroll to trigger pagination"
action: "scrollToBottom"
expected: "GET /api/v1/media?page=2 fired; new items appended to grid"
 - name: "Verify item rendering"
action: "assert [data-testid=media-card] count >= 20"
expected: "At least 20 media cards visible across both pages"
- tags: [media, pagination, ui]
estimated_duration: "20s"
expected_result: "Media grid paginates correctly; no duplicate items"
- id: WEB-003
name: "Search with Cyrillic query"
category: functional


```
priority: high
platforms: [web]
steps:
  - name: "Focus search input"
    action: "click [data-testid=search-input]"
    expected: "Input focused; placeholder text visible"
  - name: "Type Cyrillic term"
    action: "input [data-testid=search-input] фильм"
    expected: "Text 'фильм' entered; debounced search fires
      after 300ms"
  - name: "Assert results"
    action: "assert [data-testid=search-result] count > 0"
    expected: "Search results contain Cyrillic metadata; no
      mojibake"
tags: [search, i18n, cyrillic, unicode]
estimated_duration: "15s"
expected_result: "Cyrillic search returns relevant results;
  response encoding UTF-8"

- id: WEB-004
  name: "Add and remove favorite with WebSocket broadcast"
  category: functional
  priority: high
  platforms: [web]
  steps:
    - name: "Toggle favorite on first item"
      action: "click [data-testid=favorite-btn]:first"
      expected: "POST /api/v1/favorites returns 201; heart icon active"
    - name: "Verify WebSocket broadcast"
      action: "waitForWebSocketEvent favorite:updated"
      expected: "WS message received with updated favorite
        list"
    - name: "Toggle off"
      action: "click [data-testid=favorite-btn]:first"
      expected: "DELETE /api/v1/favorites/:id returns 204;
        heart icon inactive"
  tags: [favorites, websocket, realtime]
  estimated_duration: "15s"
  expected_result: "Favorite state persists and broadcasts via
    WebSocket"

- id: WEB-005
  name: "Dark mode toggle with responsive breakpoint"
  category: functional
  priority: medium
  platforms: [web]
  steps:
```



```

- name: "Set viewport to mobile"
  action: "setViewport 375x667"
  expected: "Layout switches to mobile grid"
- name: "Open settings menu"
  action: "click [data-testid=settings-menu]"
  expected: "Settings drawer visible"
- name: "Toggle dark mode"
  action: "click [data-testid=dark-mode-toggle]"
  expected:
    "html element gains class 'dark'; CSS variables updated"
- name: "Assert contrast"
  action: "assertContrastRatio [data-testid=media-card]
    4.5"
  expected: "WCAG AA contrast ratio >= 4.5:1"
tags: [ally, responsive, dark-mode, theme]
estimated_duration: "15s"
expected_result: "Dark mode applied at all breakpoints with
  accessible contrast"

```

The bank follows the HelixQA YAML schema from the upstream `admin-operations.yaml` bank^[7]: a version field, name, description, metadata block with author, app, and version, and a `test_cases` array where each case carries id, name, category, priority, platforms, steps (with name, action, expected), tags, `estimated_duration`, and `expected_result`. The action syntax uses HelixQA's domain-specific command language — `navigate`, `input`, `click`, `scrollToBottom`, `assert`, `waitForNavigation`, `waitForWebSocketEvent`, `setViewport`, and `assertContrastRatio` — primitives resolved through the chromedp-based web/CDP interactor backend¹.

6.3.2 Desktop E2E Bank: `banks/catalogizer-desktop-e2e.yaml`

This bank targets the `catalogizer-desktop` Tauri application and the `installer-wizard` SMB configuration wizard. Both are built with Tauri 2.0 (Rust backend), React 18, TypeScript, and Vite¹. The bank exercises window management, protocol connection dialogs (SMB discovery via the `Discovery` submodule), and offline/online transitions.

```

# SPDX-FileCopyrightText: 2026 Milos Vasic
# SPDX-License-Identifier: Apache-2.0

version: "1.0"
name: "Catalogizer Desktop E2E Test Bank"
description: "End-to-end tests for catalogizer-desktop and
  installer-wizard"
metadata:
  author: "vasic-digital"
  app: "Catalogizer"
  version: "2.3.0"
  clients: ["catalogizer-desktop", "installer-wizard"]
  build_tool: "tauri-cli"

```

```
test_cases:
- id: DSK-001
  name: "Tauri app launches and main window visible"
  category: e2e
  priority: critical
  platforms: [desktop_linux, desktop_macos, desktop_windows]
  steps:
    - name: "Launch binary"
      action: "exec ./target/release/catalogizer-desktop"
      expected: "Process starts; stdout contains 'ready'"
    - name: "Wait for window"
      action: "waitForWindow Catalogizer 5000"
      expected: "Window title 'Catalogizer' visible on screen"
    - name: "Capture screenshot"
      action: "screenshot /tmp/helixqa/dsk-001-launch.png"
      expected: "PNG file written; dimensions > 0x0"
  tags: [launch, window, smoke]
  estimated_duration: "10s"
  expected_result: "Application window renders within 5
    seconds"

- id: DSK-002
  name: "Installer wizard discovers SMB share"
  category: e2e
  priority: critical
  platforms: [desktop_linux, desktop_macos, desktop_windows]
  steps:
    - name: "Launch installer-wizard"
      action: "exec ./target/release/installer-wizard"
      expected: "Wizard window visible"
    - name: "Click network discovery"
      action: "click [data-testid=discover-network]"
      expected: "Discovery scan initiated; loading spinner
        visible"
    - name: "Wait for SMB result"
      action: "waitForElement [data-testid=smb-result] 30000"
      expected:
        "At least one SMB share listed with name, host, and path"
    - name: "Select first share"
      action: "click [data-testid=smb-result]:first"
      expected: "Share selected; 'Continue' button enabled"
  tags: [installer, smb, discovery, network]
  estimated_duration: "45s"
  expected_result: "SMB share discovered and selectable within
    30 seconds"

- id: DSK-003
```

```

name: "Offline mode caches API responses"
category: e2e
priority: high
platforms: [desktop_linux]
steps:
  - name: "Navigate to media page while online"
    action: "navigate /media"
    expected: "Media list loaded from API"
  - name: "Disconnect network"
    action: "exec nmcli networking off"
    expected: "Network down; app detects offline state"
  - name: "Reload media page"
    action: "navigate /media"
    expected: "Media list served from cache; no API error toast"
  - name: "Restore network"
    action: "exec nmcli networking on"
    expected: "App reconnects; WebSocket re-established"
tags: [offline, cache, resilience, network]
estimated_duration: "30s"
expected_result: "Cached data available offline; seamless reconnection"

```

The desktop bank uses `desktop_linux`, `desktop_macos`, and `desktop_windows` platform tags, matching the existing `nexus-desktop-{linux,macos,windows}` banks in HelixQA [1](#). The `exec` action spawns the Tauri binary directly; `waitForWindow`, `screenshot`, and `click` resolve through the Linux X11 or macOS/Windows native interactor backends delivered by OCU P3 Interact [1](#).

6.3.3 API Contract Bank: `banks/catalogizer-api-contract.yaml`

This bank exercises every REST endpoint in `catalog-api/` with real HTTP requests, JWT authentication, and strict response validation. The Go backend defines 22+ domain handlers under `catalog-api/handlers/` [1](#), so the bank is organized by handler category.

```

- id: API-001
  name: "JWT auth flow returns valid token"
  category: contract
  priority: critical
  platforms: [api]
  steps:
    - name: "Request token"
      action: "http POST /api/v1/auth/login {username,password}"
      expected: "200 OK; body contains token string; Expires-In header present"
    - name: "Verify token structure"

```

```

    action: "assert jwt.token matches /^[A-Za-z0-9_-]+\.[A-
    Za-z0-9_-]+\.[A-Za-z0-9_-]+$/\"
    expected: "Token is three Base64URL segments\"
tags: [auth, jwt, contract]
estimated_duration: "5s\"
expected_result: "Valid JWT with standard claims returned\"

```

The API bank uses the `api` platform tag and the `http` action primitive, which HelixQA resolves through its HTTP interactor backend against the running `catalog-api` server.

6.3.4 Android TV Bank: `banks/catalogizer-android-tv.yaml`

This bank targets `catalogizer-androidtv`. It covers channel browsing, media playback with ExoPlayer, and deep-link handling. The Android TV app uses Jetpack Compose for TV (Leanback), D-pad navigation, and the same Retrofit HTTP client as the phone app [1](#). The bank is currently blocked by the ADB device exclusion documented in §6.1.3; it is defined here so that when a non-ATMOSphere Android TV is connected, the operator has a ready-to-run bank.

```

- id: ATV-001
  name: "Browse channels with D-pad navigation\"
  category: functional
  priority: critical
  platforms: [androidtv]
  steps:
    - name: "Focus first channel row\"
      action: "adb shell input keyevent KEYCODE_DPAD_DOWN\"
      expected: "Focus highlight moves to first channel card\"
    - name: "Select channel\"
      action: "adb shell input keyevent KEYCODE_ENTER\"
      expected: "Channel detail screen visible; backdrop image
        loaded\"
  tags: [androidtv, dpad, navigation, leanback]
  estimated_duration: "20s\"
  expected_result: "D-pad navigation functional; focus
    management correct\"

```

The `androidtv` platform tag aligns with existing `full-qa-androidtv` and `nexus-mobile-android` banks [1](#). The `adb shell input keyevent` syntax is the ADB command that HelixQA's Android interactor backend translates into input events on the device.

6.3.5 Translation Workflow Bank: `banks/catalogizer-translation-workflow.yaml`

This bank covers subtitle management, language settings, and Cyrillic/Unicode text handling. Catalogizer supports multilingual metadata sourced from TMDb, MusicBrainz, and OpenLibrary, and the web client renders Cyrillic, CJK, and RTL scripts [1](#). The bank validates subtitle file parsing (SRT, ASS, VTT), language setting persistence, and Unicode string round-tripping through the API.

Table 3 consolidates the five new banks.

Table 3. New HelixQA Test Banks for Catalogizer

Bank Name	Platform	Coverage Scope	Priority	File Path
catalogizer-web-functional	Web (Chrome/CDP)	Login, media browse, Cyrillic search, favorites, dark mode, responsive breakpoints, WebSocket updates	Critical	HelixQA/banks/catalogizer-web-functional.yaml
catalogizer-desktop-e2e	Desktop (Linux/macOS/Windows)	Tauri launch, window management, SMB discovery in installer-wizard, offline/online transitions	Critical	HelixQA/banks/catalogizer-desktop-e2e.yaml
catalogizer-api-contract	API (HTTP REST)	All 22+ handlers with real JWT auth, response schema validation, rate-limit headers, WebSocket subscription lifecycle	Critical	HelixQA/banks/catalogizer-api-contract.yaml
catalogizer-android-tv	Android TV (ADB)	Channel browsing, D-pad navigation, playback, deep links, ExoPlayer state	High	HelixQA/banks/catalogizer-android-tv.yaml
catalogizer-translation-workflow	Web + API	Subtitle upload/parse (SRT/ASS/VTT), language settings persistence, Cyrillic/Unicode round-trip, RTL rendering	High	HelixQA/banks/catalogizer-translation-workflow.yaml

Priority reflects end-user criticality: web and desktop are primary daily-use surfaces; the API is the backbone for all clients; Android TV is the secondary living-room surface; and translation workflows are a differentiated feature requiring Unicode correctness. All five banks should be committed to `HelixQA/banks/` and registered in `scripts/helixqa-orchestrator.sh` [1](#).

6.4 Client App Validation

Each client requires a distinct validation strategy because the underlying interactor technology differs. HelixQA abstracts these differences through the OCU P1–P6 pipeline, but the operator must still configure the correct backend for each target [1](#).

6.4.1 Web Client (Collection-Manager-React): Playwright Automation

The web client validation uses Playwright E2E tests already present in `catalog-web/e2e/` [1](#). HelixQA integrates by invoking Playwright programmatically and attaching trace, screenshot, and video artefacts to its evidence sink. The operator adds the following test to `catalog-web/e2e/helixqa-web-smoke.spec.ts`:

```
import { test, expect } from '@playwright/test';

test.describe('HelixQA Web Smoke', () => {
  const API_BASE = process.env.VITE_API_BASE_URL || 'http://localhost:8080';

  test('login at mobile breakpoint + dark mode toggle', async ({ page }) => {
    await page.setViewportSize({ width: 375, height: 667 });
    await page.goto('/login');

    await page.fill('[data-testid="login-email"]', 'qa-user@catalogizer.local');
    await page.fill('[data-testid="login-password"]', 'qa-password-2026');
    await page.click('[data-testid="login-submit"]');
    await page.waitForURL('/dashboard', { timeout: 10000 });
    await expect(page.locator('[data-testid="media-grid"]')).toBeVisible();

    await page.click('[data-testid="settings-menu"]');
    await page.click('[data-testid="dark-mode-toggle"]');
    const htmlClass = await page.evaluate(() => document.documentElement.className);
    expect(htmlClass).toContain('dark');
  });

  test('WebSocket receives favorite update in real time', async ({ page }) => {
    const wsMessages: string[] = [];
    page.on('websocket', ws => {
      ws.on('framereceived', frame => wsMessages.push(frame.payload as string));
    });

    await page.goto('/login');
```

```

    await page.fill('[data-testid="login-email"]', 'qa-
      user@catalogizer.local');
    await page.fill('[data-testid="login-password"]', 'qa-
      password-2026');
    await page.click('[data-testid="login-submit"]');
    await page.waitForURL('/dashboard');

    await page.click('[data-testid="favorite-btn"]:first-child');
    await page.waitForTimeout(1000);

    expect(wsMessages.some(m =>
      m.includes('favorite:updated'))).toBe(true);
  });
});

```

The `data-testid` attributes are the stable selectors required by both Playwright and the HelixQA CDP interactor. The WebSocket assertion validates the real-time channel the Streaming submodule provides [1](#). The operator runs the test with:

```

cd catalog-web
npx playwright test e2e/helixqa-web-smoke.spec.ts --
  project=chromium --trace=on --video=on

```

HelixQA's `pkg/visionnav` evidence sink consumes the Playwright trace ZIP, per-test video MP4, and screenshots from `test-results/`.

6.4.2 Desktop Client (Tauri): Window Capture and Protocol Dialogs

Desktop validation uses the OCU P1 Capture and P3 Interact backends. For Linux, HelixQA uses `xwd + convert` for screenshots and `xdotool/ydotool` for input injection [1](#). The operator runs the following manual validation sequence before trusting the autonomous pipeline:

```

# Build and launch the Tauri release binary
cd catalogizer-desktop
npm run tauri build
./src-tauri/target/release/catalogizer-desktop &
PID=$!

# Verify window appearance
sleep 2
xdotool search --name "Catalogizer" | head -1
# Expected: numeric window ID, e.g., 8388609

# Capture window screenshot
xwd -id $(xdotool search --name "Catalogizer" | head -1) -out /
  tmp/catalogizer-window.xwd
convert /tmp/catalogizer-window.xwd /tmp/catalogizer-window.png
# Expected: PNG file > 0 bytes with window dimensions

# Simulate menu interaction (Ctrl+O for protocol dialog)

```



```

xdotool key --window $(xdotool search --name "Catalogizer" |
    head -1) ctrl+o
sleep 1
xdotool search --name "Open" | head -1
# Expected: numeric window ID for the "Open" dialog

kill $PID

```

This sequence confirms that `xdotool` resolves the Tauri window by title, injects keystrokes, and observes dialog spawn. Once confirmed, HelixQA's autonomous engine composes these primitives into the DSK-002 test case from `catalogizer-desktop-e2e.yaml` (§6.3.2) without further operator intervention.

6.4.3 Android Client (Kotlin): ADB-Based Navigation and Media Playback

Android validation is currently blocked, but the ADB command surface is well-defined. The operator connects a non-ATMOSphere Android phone and adds it to the `.devconnect` allowlist. The exact ADB sequence for `catalogizer-android` is:

```

# Connect and verify device
adb connect 192.168.1.105:5555
grep -q "192.168.1.105" .devconnect && echo "ALLOWED" || echo
    "BLOCKED"
# Expected: ALLOWED

# Install and launch
adb -s 192.168.1.105:5555 install -r catalogizer-android/app/
    build/outputs/apk/debug/app-debug.apk
adb -s 192.168.1.105:5555 shell am start -n
    com.vasicdigital.catalogizer/.MainActivity

# Navigate with D-pad
adb -s 192.168.1.105:5555 shell input keyevent KEYCODE_DPAD_DOWN
adb -s 192.168.1.105:5555 shell input keyevent KEYCODE_ENTER
sleep 2

# Capture screenshot for evidence
adb -s 192.168.1.105:5555 shell screencap -p /sdcard/helixqa-
    android-nav.png
adb -s 192.168.1.105:5555 pull /sdcard/helixqa-android-nav.png /
    tmp/

# Type Cyrillic search term via broadcast
adb -s 192.168.1.105:5555 shell am broadcast -a ADB_INPUT_TEXT --
    es text "фильм"

# Verify media playback via ExoPlayer state
adb -s 192.168.1.105:5555 shell input keyevent KEYCODE_DPAD_DOWN
adb -s 192.168.1.105:5555 shell input keyevent KEYCODE_ENTER
sleep 5

```

```
adb -s 192.168.1.105:5555 shell dumpsys media_session | grep -i
    "state=PlaybackState.*PLAYING"
# Expected: at least one line showing PLAYING state
```

The `adb shell am broadcast -a ADB_INPUT_TEXT` command is an alternative to the `input keyevent` stream for text entry; it relies on the Catalogizer Android app registering a broadcast receiver for the `ADB_INPUT_TEXT` action. The `dumpsys media_session` check validates ExoPlayer state without UI instrumentation, matching the pattern used by the existing [full-qa-android](#) bank [1](#).

6.4.4 API Client (Go): Contract Testing with Real HTTP Calls

The API contract validation is implemented as a Go table-driven test in `catalog-api/tests/helixqa_api_contract_test.go`. It uses the constructor-injection pattern already established in handler tests, allowing the test to spin up an in-memory SQLite-backed server or target the running Dockerized PostgreSQL stack [1](#). The test below exercises the JWT auth flow, a protected endpoint, and a WebSocket subscription lifecycle.

```
package tests
```

```
import (
    "bytes"
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "net/url"
    "strings"
    "testing"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/gorilla/websocket"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    "github.com/vasic-digital/catalog-api/handlers"
    "github.com/vasic-digital/catalog-api/models"
)

func TestHelixQA_APIContract_JWTAuthAndWebSocket(t *testing.T) {
    gin.SetMode(gin.TestMode)
    router := gin.New()

    db := setupTestDB(t)
    authHandler := handlers.NewAuthHandler(db)
    mediaHandler := handlers.NewMediaHandler(db)

    router.POST("/api/v1/auth/login", authHandler.Login)
    router.GET("/api/v1/media", authHandler.RequireAuth(),
        mediaHandler.List)
```

```

router.GET("/api/v1/ws", authHandler.RequireAuth(),
    handlers.WebSocketUpgrade)

// Step 1: Login returns valid JWT
loginBody, _ := json.Marshal(models.LoginRequest{
    Username: "qa-user", Password: "qa-password-2026",
})
req := httptest.NewRequest(http.MethodPost, "/api/v1/auth/login", bytes.NewReader(loginBody))
req.Header.Set("Content-Type", "application/json")
w := httptest.NewRecorder()
router.ServeHTTP(w, req)

require.Equal(t, http.StatusOK, w.Code)
var loginResp models.LoginResponse
require.NoError(t, json.Unmarshal(w.Body.Bytes(), &loginResp))
require.NotEmpty(t, loginResp.Token)
assert.Regexp(t, `^[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+$`, loginResp.Token)

// Step 2: Protected endpoint rejects missing token
req2 := httptest.NewRequest(http.MethodGet, "/api/v1/media?page=1", nil)
w2 := httptest.NewRecorder()
router.ServeHTTP(w2, req2)
assert.Equal(t, http.StatusUnauthorized, w2.Code)

// Step 3: Protected endpoint accepts valid token
req3 := httptest.NewRequest(http.MethodGet, "/api/v1/media?page=1", nil)
req3.Header.Set("Authorization", "Bearer "+loginResp.Token)
w3 := httptest.NewRecorder()
router.ServeHTTP(w3, req3)
assert.Equal(t, http.StatusOK, w3.Code)

// Step 4: WebSocket connection lifecycle
ts := httptest.NewServer(router)
defer ts.Close()

wsURL := "ws" + strings.TrimPrefix(ts.URL, "http") + "/api/v1/ws"
wsHeader := http.Header{}
wsHeader.Set("Authorization", "Bearer "+loginResp.Token)

ws, resp, err := websocket.DefaultDialer.Dial(wsURL, wsHeader)
require.NoError(t, err)

```

```

defer ws.Close()
assert.Equal(t, http.StatusSwitchingProtocols,
    resp.StatusCode)

subMsg := map[string]string{"action": "subscribe", "topic":
    "favorites"}
subBytes, _ := json.Marshal(subMsg)
require.NoError(t, ws.WriteMessage(websocket.TextMessage,
    subBytes))

ws.SetReadDeadline(time.Now().Add(5 * time.Second))
_, ackBytes, err := ws.ReadMessage()
require.NoError(t, err)
assert.Contains(t, string(ackBytes), "subscribed")

ws.WriteMessage(websocket.CloseMessage,
    websocket.FormatCloseMessage(websocket.CloseNormalClosure,
        "qa-done"))
}

```

The test uses `httptest.NewRecorder` and `httptest.NewServer` for deterministic local execution without requiring the Docker stack. The `setupTestDB(t)` helper, present in `catalog-api/tests/`, creates an in-memory SQLite database via `database.WrapDB(sqlDB, DialectSQLite)` [1](#). The `models.LoginRequest`, `models.LoginResponse`, and `models.MedialistResponse` types are shared entity definitions from the `Entities` submodule [1](#). The `gorilla/websocket` client validates the WebSocket upgrade path the `Streaming` submodule exposes [1](#).

The operator runs this contract test with:

```

cd catalog-api
GOMAXPROCS=3 go test ./tests/ -run
    TestHelixQA_APIContract_JWTAuthAndWebSocket -v -race
    -count=1

```

The `GOMAXPROCS=3`, `-race`, and `-count=1` flags mirror the resource-limit and race-detection discipline enforced across all Catalogizer Go tests [1](#). A PASS confirms that the API contract is stable at the new HelixQA phase, providing the regression signal required by Step 5 of the bump procedure (§6.2.5).

The integration sequence — assess, bump, extend banks, validate clients — is the operational template HelixQA applies to any multi-platform project. Catalogizer’s gaps (Android device exclusion, installer-wizard and API-client coverage holes, CUDA sidecar non-deployment) represent the friction that occurs when a QA framework outpaces operational readiness. The 206 PASS / 1 SKIP baseline, the exact commit delta from `35deb43` to `0bca023`, and the five new bank definitions provide the concrete artefacts the next chapter — the anti-bluff framework — will use to distinguish genuine functional coverage from metadata-only greenwashing.

[²]: HelixDevelopment/HelixQA commit 0bca0234bab37b5ebdb28e22fd04350b51847883 — “cascade: 1.1.5-dev: Phase 29 — §12.6 Memory-Budget Ceiling”, 2026-04-30. [³]: HelixDevelopment/DocProcessor commit 5f1e58a2289a3e3f8f125ee8612fcea12d76db6b — “test: bluff-scan annotations for must-not-panic / lifecycle / null-impl smoke tests”. [⁴]: HelixDevelopment/LLMOrchestrator commit 1b9582362d6c713d52de77bef0de087e33a24fc3 — “test: bluff-scan annotations — final pass for stress / context / race / lifecycle smokes”. [⁵]: HelixDevelopment/LLMProvider commit 0720b6e6fba0f47675cef40161ec5355d326343a — “test: bluff-scan annotations — final pass for stress / context / race / lifecycle smokes”. [⁶]: HelixDevelopment/VisionEngine commit 77765aef6d103a3d2b3358df8007205f5b35143a — “test: bluff-scan annotations for must-not-panic / lifecycle / null-impl smoke tests”. [⁷]: HelixDevelopment/HelixQA banks / admin-operations.yaml — upstream YAML bank format, commit 0bca023.